PRODUCT & ENGINEERING PLAN • 2026

Harvest Market Website Development Roadmap

A practical plan for building a multi-farmer online marketplace with a natural, earthy identity—connecting local growers with customers.

PRODUCT	Multi-farmer online marketplace
DESIGN	Natural & earthy · Responsive storefront
STARTER STACK	Next.js · React · TypeScript · Prisma · SQLite (local)
MARKET	India-ready direction: INR pricing, PIN codes, UPI-capable provider integration

The goal

Launch a dependable marketplace where customers can discover produce, learn about the farmers who grow it, place orders, pay securely, and receive deliveries—while farmers and administrators manage products, stock, and fulfilment.

Roadmap at a glance

PHASE	FOCUS	ESTIMATE	EXIT CRITERION
0	Scope & setup	2–3 days	Requirements and local dev confirmed
1	Storefront & catalog	4–6 days	Responsive browsing and basket
2	Accounts & roles	4–7 days	Secure customer/farmer/admin access
3	Marketplace operations	5–8 days	Product and order workflows
4	Payments & delivery	5–10 days	Verified payments and fulfilment flow
5	QA & launch	4–7 days	Production deployment and monitoring

Planning estimate: approximately 4–7 weeks for a small, focused team, assuming timely decisions and access to payment, hosting, and messaging-provider accounts. Provider onboarding or complex marketplace payouts may extend the schedule.

1. Current baseline & immediate next steps

A starter codebase has been prepared with a responsive storefront, sample farmers and produce, a cart, a basic order-request API, Prisma data models, and API-key-protected starter admin endpoints.

Already present in the starter

- Product catalog with categories, search, product stock, and INR prices stored as integer paise.
- Farmer profile cards and sample seed data.
- Shopping basket and delivery-details form.
- Order API with input validation, inventory checks, and database transaction logic.
- Basic product-creation and order-list admin API endpoints protected by a server-side API key.
- Responsive, natural/earthy storefront styling.

Immediate local setup

- Install Node.js 20.9 or newer and extract the starter ZIP.
- Copy `.env.example` to `.env`; set a strong private `ADMIN_API_KEY`.
- In Windows PowerShell, use `npm.cmd install`, then `npm.cmd run db:push`, `npm.cmd run db:seed`, and `npm.cmd run dev`.
- Open `http://localhost:3000` and test browsing, basket updates, and an order request.
- Keep the terminal open while testing. The current checkout records an order request; it does not take payment.

Definition of done for this stage

The app starts locally, sample data appears, the basket updates correctly, invalid checkout input is rejected, orders are stored, and stock cannot be oversold under the tested concurrent-order scenarios.

2. Product scope & user roles

Customer experience

- Browse/search/filter products by category, availability, farmer, and price.
- View product details, unit price, farmer information, stock, and seasonal availability.
- Maintain a basket; choose delivery or pickup if offered in the launch area.
- Create an order, complete payment, view order status, and receive confirmations.
- View order history and request help, cancellation, or refund under published policies.

Farmer experience

- Register and submit farm/business details for approval.
- Create/edit product listings, upload photos, set units and prices, and manage stock.
- Receive only the orders assigned to that farmer and update fulfilment status.
- Review sales, commission/fees, payout status, and customer delivery instructions as permitted.

Administrator experience

- Approve/suspend farmer accounts and moderate product listings.
- Manage categories, featured items, serviceable PIN codes, fees, and promotions.

- Review orders, payments, refunds, cancellations, customer issues, and audit events.
- Configure commission rules and reconcile farmer settlements.

Key marketplace rule

The customer basket can contain products from multiple farmers, but the system must retain farmer-level order items and fulfilment. Decide whether one checkout creates a single customer order with multiple farmer sub-orders, or separate orders per farmer. This affects delivery fees, payment settlement, refunds, and tracking.

3. Recommended technical architecture

LAYER	RECOMMENDATION	NOTES
Web app	Next.js App Router, React, TypeScript	Responsive storefront plus protected dashboards
API	Next.js route handlers or dedicated API	Validate all inputs; enforce permissions on server
Database	PostgreSQL for production; SQLite locally	Use migrations, backups, indexes and transaction-safe stock updates
ORM	Prisma	Schema migrations and typed data access
Authentication	Secure session-based auth provider/library	Customer, farmer, admin roles; MFA for administrators if feasible
Payments	Razorpay or another approved provider	Server-created payment orders, verified signatures/webhooks
Images	Managed object storage/image service	Validate file type/size; avoid local disk storage in serverless deployments
Notifications	Email and/or SMS/WhatsApp provider	Order confirmation, dispatch, delivery, cancellation/refund
Hosting	Managed Next.js hosting + managed PostgreSQL	HTTPS, environment secrets, logs, alerts and backups

Core data entities

Users & roles · Farmer profiles · Products & categories · Inventory movements · Customer orders · Farmer sub-orders/order items · Payment attempts and provider references · Delivery addresses/slots · Shipment/tracking events · Refunds · Commission and payout records · Audit logs.

4. Development phases & deliverables

Phase 0 — Scope, setup & design

Indicative duration: 2–3 days

- Confirm launch city/area, delivery radius or PIN codes, customer segment, pickup option, product categories, and operating hours.
- Define order model for multiple farmers, service fees, delivery pricing, cancellation/refund policy, and who handles customer support.
- Confirm brand name, logo direction, color palette, content, and licensed product/farmer photos.
- Run the starter locally; create a Git repository and development/staging environments.

Exit criterion: Approved scope, page map, acceptance criteria, running local starter.

Phase 1 — Storefront & product catalog

Indicative duration: 4–6 days

- Finish home, category listing, product detail, farmer detail, search/filter, basket, and responsive mobile layouts.
- Replace demo photos and content; add image upload/storage, units, stock messages, and empty/error/loading states.
- Add server-side pagination and query validation as catalog size grows.
- Add accessibility checks for keyboard navigation, labels, contrast, and responsive behavior.

Exit criterion: Customers can browse real listings and maintain a reliable basket on mobile and desktop.

Phase 2 — Authentication & permissions

Indicative duration: 4–7 days

- Implement sign-up/sign-in, password reset or trusted identity-provider flow, session handling, and account profile.
- Create customer, farmer, and admin roles; enforce authorization on every protected API and page.
- Add farmer onboarding, verification status, admin approval, and account suspension controls.
- Remove reliance on a shared admin API key for normal production administration.

Exit criterion: Role-based access tests pass; farmers cannot edit another farmer's products or orders.

Phase 3 — Marketplace operations

Indicative duration: 5–8 days

- Build farmer dashboard for product CRUD, inventory, incoming orders, and fulfilment updates.
- Build admin dashboard for farmer approval, catalog moderation, category management, and order oversight.
- Implement multi-farmer order grouping, commission calculations, order state transitions, cancellations, and audit logs.
- Use concurrency-safe inventory reservation/decrement logic; define expiry and release rules for unpaid orders.

Exit criterion: A complete order can be traced from customer basket to the correct farmer(s) and fulfilment state.

Phase 4 — Payments & delivery

Indicative duration: 5–10 days

- Create payment orders server-side using the chosen provider's supported integration.
- Verify webhook signatures and payment status server-side; make webhook processing idempotent.
- Handle failed, pending, duplicate, refunded, and disputed payments; reconcile provider records.
- Define serviceable PIN codes, delivery fees, slots/cut-off times, dispatch, proof of delivery, and failed delivery workflow.
- Test farmer payout/settlement and platform commission requirements with the provider before enabling live mode.

Exit criterion: Test payments reconcile correctly; order status cannot be forged by the browser; delivery workflow is testable.

Phase 5 — QA, security & launch

Indicative duration: 4–7 days

- Run end-to-end tests for browsing, checkout, stock races, multi-farmer baskets, payment webhooks, refunds, and order status.
- Test authorization, input validation, rate limits, secrets, dependency updates, CSRF/session protections where applicable, and data exposure.
- Configure production PostgreSQL, database migrations, backups, HTTPS, monitoring, logs, alerts, and error reporting.
- Publish privacy notice, terms, refund/cancellation policy, delivery policy, contact/support details, and applicable business/tax information.
- Launch with a small group of farmers and limited delivery coverage; monitor issues and reconcile orders daily.

Exit criterion: Production smoke tests pass; rollback and backup-restore procedures are documented; pilot is approved.

5. Payment, security & operational safeguards

- **Never trust the client for price or payment status.** Recalculate prices and totals on the server; mark an order paid only after verified provider confirmation.
- **Do not store card details.** Use the payment provider's hosted/approved checkout and follow its integration and compliance requirements.
- **Protect admin and farmer routes.** Require authenticated sessions, server-side role checks, least privilege, and audit logs.
- **Protect customer information.** Collect only what is needed, restrict staff access, define retention/deletion practices, and publish clear privacy information.
- **Make events idempotent.** Replayed webhooks and repeated checkout requests must not create duplicate payments, orders, refunds, or inventory changes.
- **Inventory and cancellation policy.** Decide when stock is reserved, how long unpaid reservations last, and how refunds/cancellations restore stock.
- **Market-specific review.** Confirm applicable Indian consumer, tax, privacy, food/product, invoicing, and marketplace obligations with a qualified local professional.

6. Launch checklist

Area	Go-live check
Business	Farmer agreements, support contact, pricing, commissions, refund and cancellation policy approved.
Catalog	Prices, units, availability, product claims and photos checked; unavailable products cannot be purchased.
Checkout	Address and PIN code validation, delivery fees, order confirmation, duplicate submission handling.
Payments	Test/live credentials separated; signatures/webhooks verified; failed and refunded payments tested.
Roles	Customer, farmer and admin permissions tested; no cross-farmer data access.
Operations	Order notifications, dispatch workflow, delivery exceptions and support ownership documented.
Infrastructure	Production database, migrations, backups, restore test, HTTPS, monitoring and secret management.
Quality	Mobile/browser checks, accessibility, performance, error states and end-to-end smoke tests.

7. Suggested milestones & decision log

Milestone A — Local demo: storefront and seeded data work. Milestone B — Private beta: sign-in, farmer dashboard, admin workflow, and order lifecycle. Milestone C — Payment sandbox: verified provider events, refunds, and reconciliation. Milestone D — Delivery pilot: serviceable PIN codes and fulfilment operations. Milestone E — Public launch: production infrastructure, support, monitoring, and operational sign-off.

Decisions to confirm early

- Initial launch geography and serviceable PIN codes.
- Who creates products: farmers directly, administrators, or both.
- Delivery versus pickup, fee structure, minimum order, delivery windows, and cut-off time.
- How multi-farmer checkout is grouped and whether customers pay once or per farmer.
- Payment provider, commission model, farmer payout schedule, and refund owner.
- Expected order volume and monthly budget for hosting, messaging, images, payment fees, and support.

Recommended immediate action

> **First, get the existing starter running locally using `npm.cmd` commands in Windows PowerShell. Then implement authentication and separate farmer/admin dashboards before integrating live payments. This sequence establishes trustworthy ownership and permissions before money and fulfilment are introduced.**

This roadmap is a planning document, not a guarantee of delivery time or legal compliance. Estimates assume a small experienced team; actual timing depends on scope, provider onboarding, testing, and business decisions.